	`</button>`
Embed JavaScript expressions in curly braces for dynamic rendering.	`<p>Hello, {name}!</p>`
Use ternary or logical operators for conditional rendering.	`{isAdmin ? <AdminPanel /> : <UserPanel />}` `{isAdmin && <p>Welcome, administrator!</p>}`

Assign unique keys when rendering lists to help React track updates efficiently.	`<ul>`
	`  {items.map((item, index) =>`
	`  <li key={index}>`
	`  {item}`
	`  </li>)}`
	`</ul>`

Use external CSS files or CSS-in-JS solutions (e.g., styled-components) for better maintainability.	`import './styles.css';`

Avoid inline styles; prefer CSS for consistent layouts.	`<button className="btn">`
	`  Click me`
	`</button>`

If necessary, use inline styles as JavaScript objects with camelCase properties.	`<button style={`
	`  { backgroundColor: 'blue',`
	`  color: 'white'`
	`  }`
	`}>`
	`  Click me`
	`</button>`

Table A.6 provides recommendations for handling whitespace in JSX to ensure consistent and predictable rendering.

Table A.6 JSX Best Practices for white space handling

Recommendation	Example

Newlines adjacent to tags are removed; newlines inside string literals are condensed into a single space.	`<div>`
	`  Hello`
	`  World`
	`</div>`
	`renders as “Hello world”.`